



PLC Error Intelligence — a **secure agent harness**

Reads a fault log → reuses past fixes → mines the IQAN forum → synthesises a **cited** solution. Every tool call passes a **security interlock** first.

Victor Otu Hayford

Industrial AI, 2026

Built for the "Build an Agent Harness" assignment.

▶ These slides are Marp (markdown → slides) — the same transform the brief mentions, and `marp` is allow-listed in the harness's own `run_command`. Render: `marp PRESENTATION.md`.

The problem

A PLC engineer hits a cryptic fault in the field:

```
XC44 ; No contact  
COUT-Suction hose 1 in/out [mA] (COUT, Pin C1:43/59) ; Open load  
VIN-Joystick 3/8" [%] (VIN, Pin C1:26) ; Low error
```

- The fix is *somewhere* on the community forum — buried across threads and comments.
- Searching by hand is slow; the same faults recur; nothing is reused.
- Sensitive sites often want it to run **offline**, with **no cloud, no API key**.

RootIQ turns that moment into a guided, cited fix — and remembers it.

What RootIQ is

Not just an app — an **agent harness**: a reusable core with sandboxed tools, a drop-in skills loader, editable config, two LLM backends, and a security interlock.

Intake	Upload PDF/image · type a code · drop a whole CSV
Reuse	Instant history cache hit (no LLM, no network)
Retrieve	Offline forum index → scrape threads + comments
Synthesise	LLM reasons across sources → cited JSON fix
Remember	Auto-saved → knowledge map of related faults
Guard	<code>security_check()</code> screens every tool call

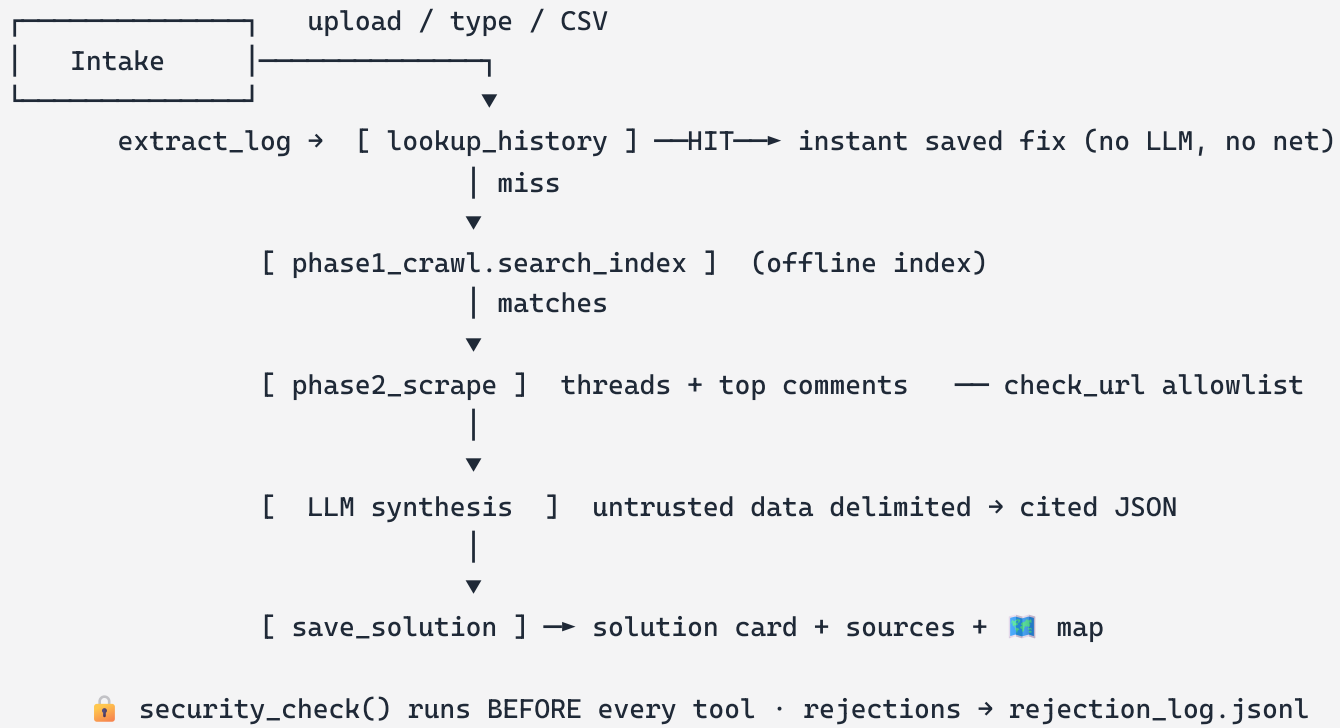
Two modes, one toggle:  offline (local **Ollama**) ·  online (any **OpenAI-compatible** API).

Why this matters — a daily bridge

This isn't a toy. I use RootIQ as the **bridge between PLC programming and software** in my day-to-day work — the controller throws faults, and this turns them into fixes.

-  **Known faults → instant reuse.** The history cache turns a recurring fault into a zero-effort, no-LLM, no-network answer. **This alone saves the most time** — the same faults come back, and now they're solved on sight.
-  **New faults → reason + search.** Unseen faults are searched against the forum index, scraped **with community comments**, and **reasoned** into a cited fix — then *remembered*, so tomorrow they're "known" too.
-  **The knowledge compounds.** Every diagnosis grows the cache and the map, so the tool gets faster and smarter the more it's used.

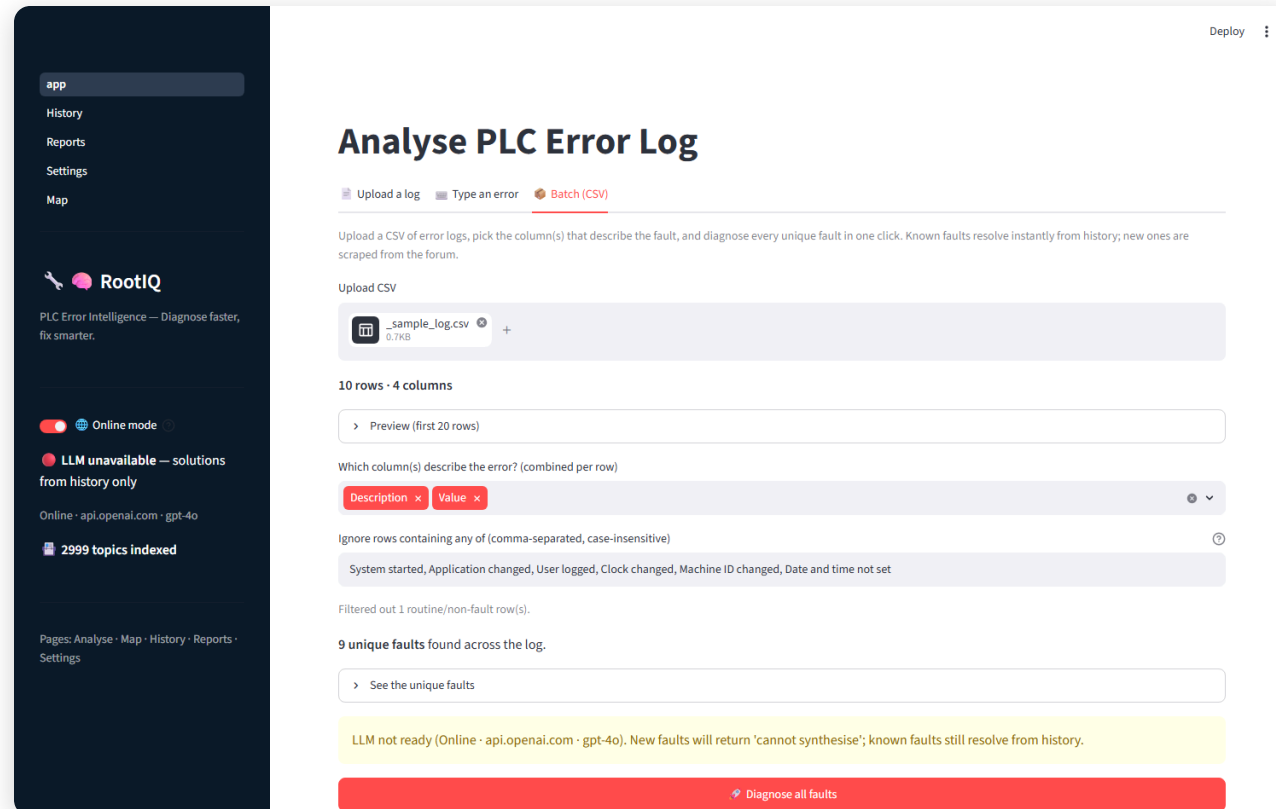
The pipeline



Demo · three ways in

The **Analyse** page has three tabs, all feeding the same pipeline & result card:

-  **Upload a log** — PDF/image; codes auto-detected as badges
-  **Type an error** — paste a code or a fault sentence, no file
-  **Batch (CSV)** — a controller export; de-duped to **unique faults**, diagnosed in one click



Demo · a cited, multi-source answer

For each fault, RootIQ returns:

- **Root cause** + **numbered fix steps** with inline citations [1] [2]
- **Confidence** badge · **From history** / **From forum** source
- **Every source link** it reasoned from — one or many

It **reasons across multiple threads and their highest-voted comments**, not one — and never parrots a single page.

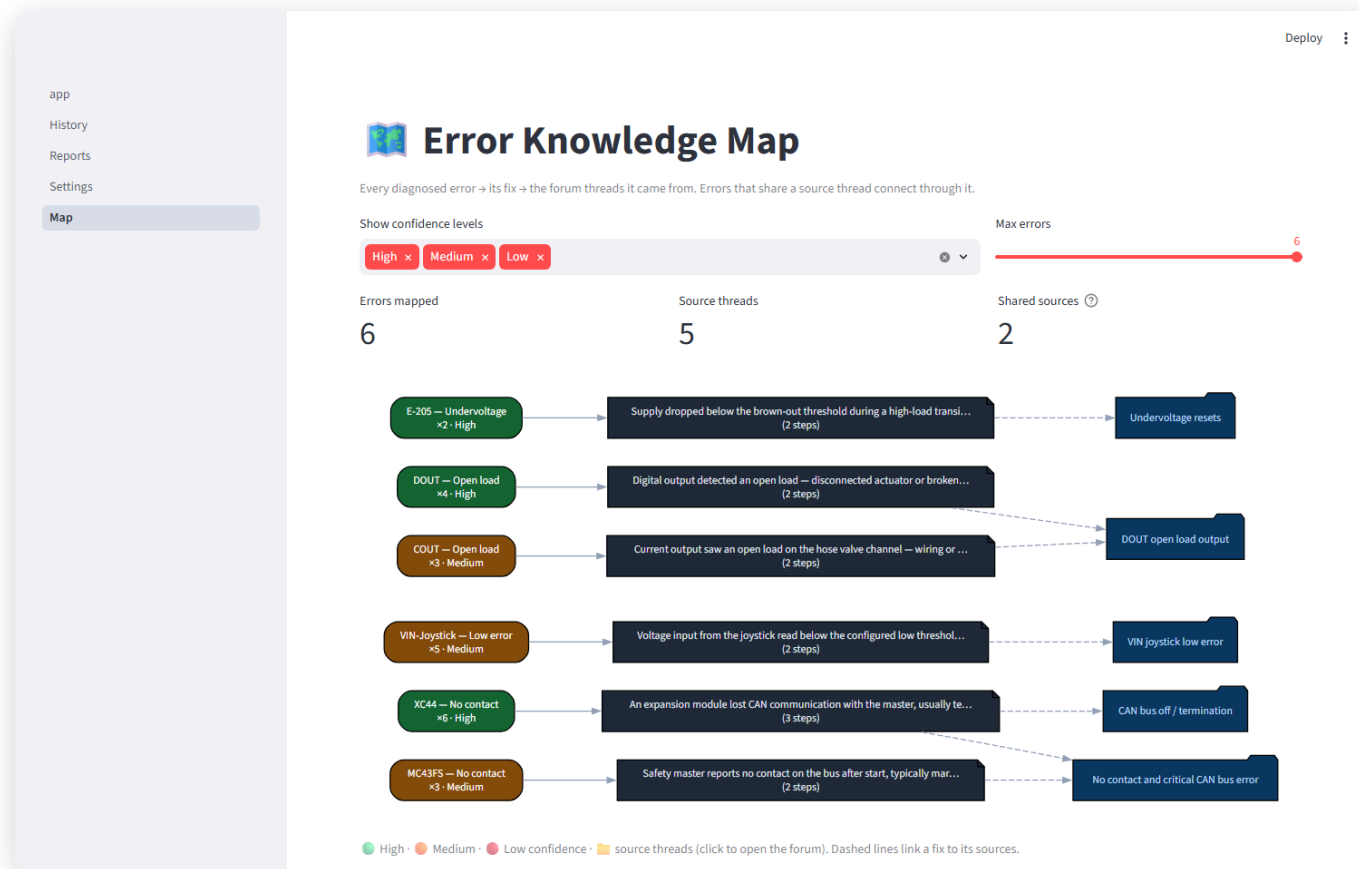
Output is parsed **only** as a JSON solution object — the model is **not** wired to any file/command tool, so a hijacked reply can't reach a dangerous action.

Demo · the knowledge map

 **Map** page — a Graphviz graph over every diagnosis:

- each **unique error** is one node (sized by occurrence)
- linked to its **fix** and the **forum threads** it cited
- **shared source threads converge** → related faults visibly connect

Plus **History** (searchable, one-click reset) and **Reports** (top codes, source split, trend).



The harness — tools, skills & config

Sandboxed tools (`tools.py`) — every path forced through `_safe_path()`: `read_file` · `write_file` · `create_markdown` · `list_files` · **`transform_markdown`** (md→HTML) · `run_command` (allow-listed).

Drop-in skills (`skills/`) — `load_skills()` auto-discovers any `*.py` with a `SKILL` dict. **Drop one in, the agent picks it up — no harness edit.**

skill	role
<code>extract_log</code>	problem log (PDF/image) → text + codes — <i>the loaded extension</i>
<code>lookup_history</code> / <code>save_solution</code>	the offline reuse cache
<code>phase1_crawl</code> / <code>phase2_scrape</code>	two-phase forum retrieval

Editable config (`mcp.json`) — LLM provider, forums, scraping limits; no code change.

Offline and online — by design

Mode	LLM	Use
 Offline	local Ollama (<code>localhost:11434</code>)	air-gapped, no key, "real work with the network unplugged"
 Online	any OpenAI-compatible API	hosted demo (Streamlit Cloud), <code>gpt-4o</code> / Groq / OpenRouter

- One sidebar toggle; choice persists to `mcp.json`.
- The **API key is never typed in, never in `mcp.json` / `git`** — read from Streamlit secrets / env at call time, used only in the `Authorization` header.
- Offline mode has **no key at all** — nothing to leak.

Security — the core of the brief

Defense in depth, four layers:

1. `security_check(tool, args)` — regex + command screen before every tool
2. `_safe_path()` — **authoritative** sandbox (`resolve()` + `is_relative_to`)
3. `check_url()` — scheme check + domain allowlist before any HTTP
4. `rejection_log.jsonl` — every block recorded, shown in Settings

```
security_check("run_command", {"command": "rm -rf /"}) # → (False, 'Blocked command: rm')
check_url("https://evil.com/x")                      # → (False, 'Domain not in allowlist')
_safe_path("../..etc/passwd")                       # → raises SandboxError
```

Security — per-tool verdicts

Tool / surface	Risk	Verdict
File ops	path traversal	 Blocked — <code>_safe_path</code> sandbox
<code>run_command</code>	shell injection	 Mitigated — <code>shell=False</code> + allow/deny lists
<code>transform_markdown</code>	HTML/script injection	 Mitigated — escape + tag whitelist
Outbound HTTP	SSRF / abuse	 Mitigated — <code>check_url</code> allowlist
Prompt injection	scraped/file text	 Mitigated by design — data delimited; output JSON-only, not wired to tools
Secrets	key leakage	 Mitigated — secrets/env only, never logged/prompted
Skills in <code>skills/</code>	arbitrary import-time code	 Accepted — first-party code, documented

Honest acceptance, not pretending the risk is absent.

How it maps to the assignment

Deliverable	✓ Status
1 · The harness (working code)	All modules compile; app runs headless, zero exceptions
2 · A loaded extension	5 drop-in skills, headline = <code>extract_log</code>
3 · README + security discussion	Per-tool risks → blocked / mitigated / accepted
4 · Presentation + screenshots	<i>this deck</i> + <code>working_dir/screenshots/</code>

Inspiration ideas hit: offline (Ollama) · safety interlock + log + allowlist · eval/retry loop (strict JSON reprompt) · context caps · staged multi-agent roles.

Honesty notes (the brief rewards this)

- `mcp.json` is config, not a literal MCP server. The real extension mechanism is the drop-in `skills/` loader — stated, not hidden.
- **Skills are not sandboxed** — trusted first-party code; importing one = editing the app.
- `python` is allow-listed for `run_command` (capable) — accepted convenience.
- **No `robots.txt` parsing** — we rate-limit, identify our UA, read only public threads.
- **Hosted history is ephemeral** — Streamlit Cloud resets on reboot; persistence needs a DB.

Run it

```
pip install -r requirements.txt

# Offline:  ollama serve && ollama pull phi3    → toggle 🌐 OFF
# Online:   export OPENAI_API_KEY=sk- ...       → toggle 🌐 ON
streamlit run app.py                          # http://localhost:8501
```

RootIQ — diagnose faster, fix smarter. Securely.

Code: `app.py`, `agent.py`, `tools.py`, `security.py`, `skills/`, `mcp.json` · *README has the full security discussion + diagrams.*

 **Thank you**

RootIQ — PLC Error Intelligence

Victor Otu Hayford · Industrial AI, 2026

Questions welcome.